

sken_library 完全版仕様書 (サンプルコード付き)

1. system クラス (SYSTEM)

マイコンの基本的な時間関数、タイマー割り込み、CAN通信の制御を行います。

構造体: `Can_data`

要素名	型	説明
<code>my_stdid</code>	<code>uint32_t</code>	自身のCANスタンダードID
<code>rx_stdid</code>	<code>uint32_t</code>	受信したメッセージの送信元ID
<code>rx_data[8]</code>	<code>uint8_t[8]</code>	受信データ(8バイト)
<code>tx_data[8]</code>	<code>uint8_t[8]</code>	送信データ(8バイト)

```
struct Can_data{  
  
    uint32_t my_stdid;  
  
    uint32_t rx_stdid;  
  
    uint8_t rx_data[8];  
  
    uint8_t tx_data[8];  
  
};
```

メソッド一覧

```
void SYSTEM::init(bool use_external_crystal = false)
```

****初期化関数。****プログラムで最初に呼び出す必要があります。

パラメータ	説明
<code>use_external_crystal</code>	外部クロック使用 (<code>true/false</code>)

```
#include "sken_library/include.h"
```

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    while(1)
```

```
    {
```

```
    }
```

```
}
```

```
unsigned int SYSTEM::micros(void)
```

マイコン起動時からの時間をマイクロ秒で返します。

戻り値	説明
<pre>unsigned int</pre>	マイコン起動時からの時間

```
unsigned int time;
```

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    while(1)
```

```
    {
```

```
        time = sken_system.micros(); // マイクロ秒を取得
```

```
    }
```

```
}
```

unsigned int SYSTEM::millis(void)

マイコン起動時からの時間をミリ秒で返します。

戻り値	説明
unsigned int	マイコン起動時からの時間

```
unsigned int time;
```

```
int main(void)
```

```
{  
  
    sken_system.init();  
  
    while(1)  
  
    {  
  
        time = sken_system.millis(); // ミリ秒を取得  
  
    }  
  
}
```

void SYSTEM::delayMillis(unsigned int delay_time_millis)

プログラムを指定した時間(ミリ秒)だけ停止させます。

パラメータ	説明
delay_time_millis	停止時間 [ms]

```
int main(void)
```

```
{  
  
    sken_system.init();
```

```
sken_system.delayMillis(1000); // 1秒待ってからループに入る

while(1)

{

}

}
```

```
void SYSTEM::delayMicros(unsigned int delay_time_micros)
```

プログラムを指定した時間(マイクロ秒)だけ停止させます。

パラメータ	説明
<code>delay_time_micros</code>	停止時間 [μs]

```
int main(void)

{

    sken_system.init();

    sken_system.delayMicros(1000); // 1ミリ秒(1000μs)待ってからループに入る

    while(1)

    {

    }

}
```

```
bool SYSTEM::addTimerInterruptFunc(void(*function_p)(void),int
id,int period = 1)
```

タイマー割り込み関数を追加します。

パラメータ	説明
<code>function_p</code>	割り込み時に呼び出す関数のアドレス
<code>id</code>	関数のID(0～7)
<code>period</code>	呼び出し周期 [ms] (デフォルト1ms)
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

```

unsigned int counter;

void func(void)
{
    counter++; // 1msごとにインクリメントされる
}

int main(void)
{
    sken_system.init();

    // ID 0, 周期 1ms で func を登録
    sken_system.addTimerInterruptFunc(func, 0, 1);

    while(1)
    {

    }
}

```

`bool SYSTEM::deleteTimerInterruptFunc(int id)`

指定したIDのタイマー割り込み関数を削除します。

パラメータ	説明
<code>id</code>	削除する関数のID (0~7)
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

// ... func と counter は addTimerInterruptFunc のサンプルを参照

```
int main(void)
{
    sken_system.init();

    sken_system.addTimerInterruptFunc(func, 0, 1);

    while(1)
    {
        if(counter > 1000){
            sken_system.deleteTimerInterruptFunc(0); // 1000ms後に割り込みを停止
        }
    }
}
```

`bool SYSTEM::changeTimerInterruptPeriod(int id,int period)`

指定したIDのタイマー割り込み関数の周期を変更します。

パラメータ	説明
<code>id</code>	周期を変更する関数のID (0~7)
<code>period</code>	新しい周期 [ms]

パラメータ	説明
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

// ... func と counter は addTimerInterruptFunc のサンプルを参照

```
int main(void)
{
    sken_system.init();

    sken_system.addTimerInterruptFunc(func, 0, 1);

    while(1)
    {
        if(counter > 1000){

            // 1000msを超えたら周期を 2ms に変更

            sken_system.changeTimerInterruptPeriod(0, 2);

        }
    }
}
```

```
bool SYSTEM::startCanCommunicate(PIN tx_pin, PIN
rx_pin, CAN_SELECT can_select)
```

CAN通信 (1Mbps)を開始します。

パラメータ	説明
<code>tx_pin</code>	CAN_TXピン
<code>rx_pin</code>	CAN_RXピン

パラメータ	説明
<code>can_select</code>	CAN番号 (<code>CAN_1</code> か <code>CAN_2</code>)
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

```
int main(void)
{
    sken_system.init();

    sken_system.startCanCommunicate(B9, B8, CAN_1); // CAN1で通信開始

    while(1)
    {

    }
}
```

```
bool SYSTEM::canTrancemit(CAN_SELECT can_select,uint32_t
stdid,uint8_t* data_p,int data_size,int dead_time = 10)
```

CAN通信でデータを送信します。

パラメータ	説明
<code>can_select</code>	CAN番号 (<code>CAN_1</code> か <code>CAN_2</code>)
<code>stdid</code>	スタンダードID
<code>data_p</code>	送信データアドレス
<code>data_size</code>	送信データ数
<code>dead_time</code>	デッドタイム [ms] (デフォルト10ms)
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

```
uint8_t can_transmit_data_1[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
```

```

int main(void)

{

    sken_system.init();

    sken_system.startCanCommunicate(B9, B8, CAN_1);

    while(1)

    {

        // CAN1, ID 0x1010 で 8バイトのデータを送信

        sken_system.canTrancemit(CAN_1, 0x1010, can_transmit_data_1, 8);

    }

}

```

```

bool SYSTEM::addCanRceiveInterruptFunc(CAN_SELECT
can_select, Can_data* can_data)

```

CAN通信で受信割り込み関数を実行します。

パラメータ	説明
<code>can_select</code>	CAN番号 (<code>CAN_1</code> か <code>CAN_2</code>)
<code>can_data</code>	CAN通信用構造体のアドレス
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

```

CanData can_data;

```

```

int main(void)

```

```

{

    sken_system.init();

```

```

sken_system.startCanCommunicate(B9, B8, CAN_1);

// CAN1の受信データを can_data に格納

sken_system.addCanRceiveInterruptFunc(CAN_1, &can_data);

while(1)

{

}

}

```

bool SYSTEM::deleteCanRceiveInterruptFunc(CAN_SELECT can_select)

CAN通信で受信割り込み関数を削除します。

パラメータ	説明
can_select	CAN番号 (CAN_1かCAN_2)
戻り値	成功で true 、失敗で false

```

CanData can_data;

int main(void)

{

    sken_system.init();

    sken_system.startCanCommunicate(B9, B8, CAN_1);

    sken_system.addCanRceiveInterruptFunc(CAN_1, &can_data);


    sken_system.delayMillis(1000); // 1秒待機

```

```
sken_system.deleteCanRceiveInterruptFunc(CAN_1); // 受信割り込みを削除

while(1)

{

}

}
```

2. 📡 uart クラス (Uart)

UART通信を制御します。ポーリングモードでの送受信とDMAを使用した受信が可能です。

メソッド一覧

```
void Uart::init(Pin tx_pin, Pin rx_pin, UartNumber uart_num, int
baudrate, uint32_t parity = UART_PARITY_NONE)
```

シリアル通信の初期化を行います。

パラメータ	説明
<code>tx_pin</code>	送信ピン番号
<code>rx_pin</code>	受信ピン番号
<code>uart_num</code>	シリアル番号
<code>baudrate</code>	転送レート [bps]
<code>parity</code>	パリティ設定 (デフォルトUART_PARITY_NONE)

```
Uart serial;
```

```
int main(void)
```

```

{

    sken_system.init();

    serial.init(A9, A10, SERIAL1, 9600); // UART1を9600bpsで設定

    while(1)

    {

    }

}

```

```

void Uart::startDmaRead(uint8_t* data_p,int data_size)

```

データ受信をDMAで行うための設定を行います。

パラメータ	説明
<code>data_p</code>	格納先配列の先頭アドレス
<code>data_size</code>	格納先配列の大きさ

```

Uart serial;

uint8_t data[2];

int main(void)

{

    sken_system.init();

    serial.init(A9, A10, SERIAL1, 9600);

    serial.startDmaRead(data, 2); // 2バイトのデータをDMAで受信

    while(1)

    {

```

```
}  
  
}
```

`uint8_t Uart::read(unsigned int dead_time)`

ポーリングモードでデータを受信します。タイムアウトフラグは`checkTimeOut()`で確認します。

パラメータ	説明
<code>dead_time</code>	受信待ち時間 [ms]
戻り値	受信データ (1バイト)

```
Uart serial;  
  
uint8_t data;  
  
int main(void)  
{  
  
    sken_system.init();  
  
    serial.init(A9, A10, SERIAL1, 9600);  
  
    while(1)  
    {  
  
        data = serial.read(1000); // 1000ms(1秒)待機して受信  
  
    }  
  
}
```

```
void Uart::write(uint8_t* data_p,int data_size,int dead_time = 100)
```

ポーリングモードでデータを送信します。

パラメータ	説明
<code>data_p</code>	送信データ配列の先頭アドレス
<code>data_size</code>	送信データ配列の大きさ
<code>dead_time</code>	デッドタイム [ms] (デフォルト100ms)

```
Uart serial;
```

```
uint8_t data[2] = {1, 2};
```

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    serial.init(A9, A10, SERIAL1, 9600);
```

```
    while(1)
```

```
    {
```

```
        serial.write(data, 2); // 2バイトのデータを送信
```

```
    }
```

```
}
```

```
bool Uart::checkTimeOut(void)
```

ポーリングモードでデータを受信した際のタイムアウトフラグを確認できます。

戻り値	説明
bool	タイムアウトしていればtrue

// ... serial, data は read のサンプルを参照

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    serial.init(A9, A10, SERIAL1, 9600);
```

```
    while(1)
```

```
    {
```

```
        data = serial.read(1000);
```

```
        if(serial.checkTimeOut()){ // タイムアウトした場合
```

```
            // 通信切断処理など
```

```
        }
```

```
        else{
```

```
            // 受信成功処理
```

```
        }
```

```
    }
```

```
}
```

3. ⚡ gpio クラス (Gpio) / PWM

GPIOピンの入出力およびPWM出力を扱います。

列挙体: Mode

定数	説明
OUTPUT	デジタル出力
INPUT	デジタル入力
INPUT_PULLUP	デジタル入力 (プルアップ)
PWM_OUTPUT	PWM出力

列挙体: PinState

定数	説明
LOW	LOW (0)
HIGH	HIGH (1)

メソッド一覧

```
void Gpio::init(Pin pin, Mode pin_mode)
```

デジタル入出力の設定を行います。

パラメータ	説明
pin	ピン番号
pin_mode	入出力の設定 (OUTPUT, INPUT, INPUT_PULLUP)

```
Gpio gpio;
```

```
int main(void)
```

```
{
```

```

sken_system.init();

gpio.init(A5, OUTPUT); // A5ピンをデジタル出力に設定

while(1)

{

}

}

```

```

void Gpio::init(Pin pin, Mode pin_mode, TimerNumber
tim_num, TimerChannel tim_ch, int frequency = 16800)

```

PWM出力の設定を行います。

パラメータ	説明
<code>pin</code>	ピン番号
<code>pin_mode</code>	<code>PWM_OUTPUT</code>
<code>tim_num</code>	タイマー番号
<code>tim_ch</code>	タイマーチャンネル
<code>frequency</code>	PWM周波数 [Hz] (デフォルト16800Hz)

```

Gpio gpio;

int main(void)

{

    sken_system.init();

    gpio.init(A5, PWM_OUTPUT, TIMER2, CH1); // A5ピンをPWM出力に設定

    while(1)

    {

```

```
}  
  
}
```

```
void Gpio::write(PinState state)
```

デジタル出力設定ピンからHIGHまたはLOWを出力します。

パラメータ	説明
<code>state</code>	出力状態 (HIGHまたはLOW)

```
Gpio gpio;  
  
int main(void)  
{  
  
    sken_system.init();  
  
    gpio.init(A5, OUTPUT);  
  
    while(1)  
    {  
  
        gpio.write(HIGH); // A5ピンからHIGHを出力  
  
    }  
  
}
```

```
void Gpio::write(double percentage)
```

PWM出力設定ピンから矩形波を出力します。

パラメータ	説明
percentage	出力(デューティ比) [%] (0.0～100.0)

```
Gpio gpio;

int main(void)
{
    sken_system.init();

    gpio.init(A5, PWM_OUTPUT, TIMER2, CH1);

    while(1)
    {
        gpio.write(50); // A5ピンから50%のPWMを出力
    }
}
```

bool Gpio::read(void)

デジタル入力設定ピンの値を読み取ります。

戻り値	説明
bool	HIGHのときtrue、LOWのときfalse

```
Gpio gpio;

int main(void)
{
    sken_system.init();
```

```

gpio.init(C13, INPUT_PULLUP); // C13ピンを入力(プルアップ)に設定

while(1)

{

    if(!gpio.read()){ // スイッチが押された(LOWになった)場合

        // 処理

    }

}

}

```

4. 🧠 pid クラス (Pid)

PID制御を行います。

メソッド一覧

```
void Pid::setGain(double p_gain, double i_gain, double
d_gain, double time_constant = 0)
```

PIDゲインの設定を行います。

パラメータ	説明
<code>p_gain</code>	比例制御ゲイン
<code>i_gain</code>	積分制御ゲイン
<code>d_gain</code>	微分制御ゲイン
<code>time_constant</code>	微分前ローパスフィルタの時定数 [ms] (デフォルト0)

```
Pid pid_control;
```

```

int main(void)

{

    sken_system.init();

    // P=1, I=1, D=1, 時定数20ms で設定

    pid_control.setGain(1, 1, 1, 20);

    while(1)

    {

    }

}

```

```
double Pid::control(double target,double now,int control_period)
```

目標値と現在値からPID制御を行います。

パラメータ	説明
<code>target</code>	目標値
<code>now</code>	現在値
<code>control_period</code>	制御周期 [ms]
戻り値	制御出力

```

Pid pid_control;

double target = 100.0;

double now = 0.0;

double out;

void func(void)

```

```

{

    out = pid_control.control(target, now, 1); // 周期 1ms で制御

    // now を更新する処理

}

int main(void)

{

    sken_system.init();

    pid_control.setGain(1, 1, 1);

    sken_system.addTimerInterruptFunc(func, 0, 1);

    while(1)

    {

    }

}

```

```
double Pid::control(double e,int control_period)
```

偏差からPID制御を行います。

パラメータ	説明
e	偏差 ϵ
control_period	制御周期 [ms]
戻り値	制御出力

```
Pid pid_control;
```

```

double e; // 外部で設定される偏差

double out;

void func(void)

{
    out = pid_control.control(e, 1); // 周期 1ms で制御
}

int main(void)

{
    sken_system.init();

    pid_control.setGain(1, 1, 1);

    sken_system.addTimerInterruptFunc(func, 0, 1);

    while(1)

    {

    }

}

```

```

void Pid::reset(void)

```

積分項と微分項をリセットします。

```

// ... pid_control は control のサンプルを参照

```

```

// ... func は control のサンプルを参照

```

```

int main(void)

```

```

{

```

```

sken_system.init();

pid_control.setGain(1, 1, 1);

sken_system.addTimerInterruptFunc(func, 0, 1);

while(1)

{

    if(sken_system.millis() % 1000 == 0){

        pid_control.reset(); // 1秒ごとにPID制御をリセット

    }

}

}

```

5. encoder クラス (Encoder)

インクリメンタル式ロータリーエンコーダのパルスをカウントします。

構造体: **Encoder_data**

要素名	型	説明
count	int	カウンタ値 (-30000～30000)
rot, deg, distance, volcity, rps	double	回転数、角度[°]、移動距離 [mm]、速度[mm/s]、回転速度 [rps]

メソッド一覧

`void Encoder::init(Pin a_pin, Pin b_pin, TimerNumber tim_num)`

エンコーダのピンとタイマー番号を設定します。

パラメータ	説明
<code>a_pin</code>	チャンネル1のピン番号
<code>b_pin</code>	チャンネル2のピン番号
<code>tim_num</code>	タイマー番号 (1, 2, 3, 4, 5, 8)

```
Encoder encoder;
```

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    encoder.init(A0, A1, TIMER2); // タイマー2をエンコーダモードに設定
```

```
    while(1)
```

```
    {
```

```
    }
```

```
}
```

`int Encoder::read(void)`

カウンタの値を取得します。

戻り値	説明
<code>int</code>	カウンタ値

```

Encoder encoder;

int count;

int main(void)
{
    sken_system.init();

    encoder.init(A0, A1, TIMER2);

    while(1)
    {
        count = encoder.read(); // カウンタ値を取得
    }
}

```

```

void Encoder::interrupt(Encoder_data* encoder_data)

```

タイマー割り込み関数内で使用し、`Encoder_data`構造体にエンコーダからの取得値(速度情報含む)を格納します。

パラメータ	説明
<code>encoder_data</code>	<code>Encoder_data</code> 構造体のアドレス

```

Encoder encoder;

Encoder_data e_data;

void encoder_interrupt(){

    encoder.interrupt(&e_data); // 割り込みで取得値を格納

}

```

```
int main(void){

    sken_system.init();

    encoder.init(A0, A1, TIMER2);

    sken_system.addTimerInterruptFunc(encoder_interrupt, 0, 1); // 1ms周期で実行

    while(1)

    {

    }

}
```

void Encoder::reset(void)

カウンタの値をリセットします。

```
Encoder encoder;

int count;

int main(void)

{

    sken_system.init();

    encoder.init(A0, A1, TIMER2);

    while(1)

    {

        count = encoder.read();

        if(count > 10000){

            encoder.reset(); // 10000を超えた時にリセット
```

```
    }  
}  
}
```

6. rc_pwm クラス (RcPwm)

周期20[ms]のラジコンサーボ信号を制御するクラスです。

メソッド一覧

```
void RcPwm::init(Pin pin, TimerNumber tim_num, TimerChannel  
tim_ch)
```

ピンとタイマーの設定を行います。

パラメータ	説明
pin	ピン番号
tim_num	タイマー番号
tim_ch	タイマーチャンネル

```
RcPwm rc_pwm;
```

```
int main(void)
```

```
{  
    sken_system.init();  
  
    rc_pwm.init(A5, TIMER2, CH1); // A5ピンをRC-PWM出力に設定  
  
    while(1)  
    {  
  
    }  
}
```

`void RcPwm::turn(double percentage)`

出力の割合を設定します。パルス幅1000[us]から2000[us]の間を0[%]から100[%]で設定します。

パラメータ	説明
<code>percentage</code>	出力割合 [%] (0.0～100.0)

```
RcPwm rc_pwm;
```

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    rc_pwm.init(A5, TIMER2, CH1);
```

```
    while(1)
```

```
    {
```

```
        rc_pwm.turn(20); // 20% (1200[us]) で出力
```

```
    }
```

```
}
```

7. gyro クラス (Gyro)

R6093ジャイロを扱うクラスです。

列挙体: `Cordinate`

定数	説明
<code>ROLL</code>	ロール軸

定数	説明
PITCH	ピッチ軸
YAW	ヨー軸

構造体: `gyro_data_t`

要素	説明
<code>roll.angle</code>	ロール角度
<code>roll.rate</code>	ロール角速度
<code>roll.acceleration</code>	ロール加速度(生値)
<code>pitch, yaw</code>	同様の要素

メソッド一覧

```
void Gyro::init(Pin tx_pin, Pin rx_pin, UartNumber uart_num, int
connect_dead_time = 1000)
```

シリアルピンの設定を行います。

パラメータ	説明
<code>tx_pin</code>	送信ピン番号
<code>rx_pin</code>	受信ピン番号
<code>uart_num</code>	シリアル番号
<code>connect_dead_time</code>	通信切断時間 [ms] (デフォルト1000ms)

```
Gyro gyro;
```

```
int main(void)
```

```
{
```

```
    sken_system.init();
```

```
    gyro.init(C12, D2, SERIAL5); // SERIAL5でジャイロ通信を初期化
```

```
while(1)

{

}

}
```

bool Gyro::update(void)

ジャイロからのデータを更新します。

戻り値	説明
bool	成功でtrue、失敗でfalse

```
Gyro gyro;

int main(void)

{

    sken_system.init();

    gyro.init(C12, D2, SERIAL5);

    while(1)

    {

        gyro.update(); // データを更新

    }

}
```

bool Gyro::resetAngle(Cordinate coordinate)

ジャイロの角度を0度にリセットします。

パラメータ	説明
<code>coordinate</code>	リセットする軸 (<code>ROLL</code> , <code>PITCH</code> , <code>YAW</code>)
戻り値	成功で <code>true</code> 、失敗で <code>false</code>

```
Gyro gyro;

int main(void)
{
    sken_system.init();

    gyro.init(C12, D2, SERIAL5);

    while(1)
    {
        gyro.update();

        if(sken_system.millis() % 1000 == 0){
            gyro.resetAngle(ROLL); // 1秒ごとにロール軸をリセット
        }
    }
}
```

`bool Gyro::getConnectFlag(void)`

接続確認フラグを取得します。

戻り値	説明
<code>bool</code>	接続していれば <code>true</code> 、そうでなければ <code>false</code>

```

Gyro gyro;

int main(void)

{

    sken_system.init();

    gyro.init(C12, D2, SERIAL5);

    while(1)

    {

        gyro.update();

        if(gyro.getConnectFlag()){

            // 接続中の処理

        }

        else{

            // 非接続時の処理

        }

    }

}

```

gyro_data_t Gyro::getData(void)

角度、角速度、加速度のデータを取得します。

戻り値	説明
gyro_data_t	ジャイロデータ構造体

Gyro gyro;

```
gyro_data_t gyro_data;

int main(void)

{

    sken_system.init();

    gyro.init(C12, D2, SERIAL5);

    while(1)

    {

        gyro.update();

        gyro_data = gyro.getData(); // データを取得

    }

}
```

8. PS3 クラス (PS3)

PS3コントローラからの受信を制御するためのクラスです。

構造体: **PS3_data**

要素	説明
Up, Down, ... Select	各ボタンの状態(押下時 1 、非押下時 0)
LxPad, LyPad, RxPad, RyPad	ジョイスティックの値(± 64)
rx_data[8]	受信生データ

メソッド一覧

```
void PS3::StartRecive(Pin tx_pin = A9,Pin rx_pin =  
A10,UartNumber uart_num = SERIAL1)
```

PS3コントローラからの受信を開始します。

パラメータ	説明
<code>tx_pin, rx_pin, uart_num</code>	送受信ピンおよびシリアル番号(デフォルト値は部室シールド基盤用)

```
PS3 ps3;
```

```
int main(void){  
  
    sken_system.init();  
  
    ps3.StartRecive(); // デフォルト設定で受信開始  
  
    while(true){  
  
    }  
  
}
```

```
bool PS3::Getdata(PS3_data* ps3_data)
```

PS3コントローラのデータを取得します。

パラメータ	説明
<code>ps3_data</code>	<code>PS3_data</code> 構造体のアドレス
戻り値	受信データを取得できれば <code>true</code> 、できなければ <code>false</code>

```
PS3 ps3;
```

```
PS3_data ps3_data;
```

```
int main(void){

    sken_system.init();

    ps3.StartRecive();

    while(true){

        if(ps3.Getdata(&ps3_data)){

            // データが更新された場合の処理

        }

    }

}
```

9. 🏠 Motor クラス (Motor)

モーターを制御するためのクラスです。

メソッド一覧

```
void Motor::init(Pin pin_a,Pin pin_p,Pin pin_b,TimerNumber
timer,TimerChannel ch)
```

芝ドラのピン、タイマー、チャンネルを設定します。

パラメータ	説明
<code>pin_a, pin_p, pin_b</code>	Aピン, PWMピン, Bピン
<code>timer, ch</code>	タイマー番号, タイマーチャンネル

```
Motor motor;
```

```
int main(void){
```

```
    sken_system.init();
```

```

// C2:Aピン, C8:PWMピン, C3:Bピン を設定

motor.init(C2, C8, C3, TIMER8, CH3);

while(true){

}

}

```

```

void Motor::init(MtrPin mtr_pin, Pin pin, TimerNumber
timer, TimerChannel ch)

```

アルタイルドライバーのピン、タイマー、チャンネルを設定します。

パラメータ	説明
<code>mtr_pin</code>	<code>Apin</code> もしくは <code>Bpin</code>
<code>pin</code>	ピン番号
<code>timer, ch</code>	タイマー番号, タイマーチャンネル

```

Motor motor;

int main(void){

    sken_system.init();

    motor.init(Apin, A0, TIMER5, CH1); // A0をAピンに設定

    motor.init(Bpin, A1, TIMER5, CH2); // A1をBピンに設定

```

```
while(true){  
  
}  
  
}
```

```
void Motor::write(int val)
```

モーターを回します。

パラメータ	説明
<code>val</code>	PWM値 (-100～100、負の数は逆回転)

```
Motor motor;
```

```
int main(void){
```

```
    sken_system.init();
```

```
    motor.init(Apin, A0, TIMER5, CH1);
```

```
    motor.init(Bpin, A1, TIMER5, CH2);
```

```
    while(true){
```

```
        motor.write(30); // モータをPWM値30で正回転
```

```
    }
```

```
}
```

```
void Motor::stop()
```

モータを停止させます。

```
Motor motor;
```

```
int main(void){

    sken_system.init();

    motor.init(Apin, A0, TIMER5, CH1);

    motor.init(Bpin, A1, TIMER5, CH2);


    while(true){

        motor.stop(); // モータを停止させる

    }

}
```

10. IM920 クラス (IM920)

通信モジュール、IM920sLを使用するためのクラスです。

列挙体: **IM920Bytes**

定数	説明
Bytes8	8バイトの送受信
Bytes16	16バイトの送受信
Bytes24	24バイトの送受信
Bytes32	32バイトの送受信

メソッド一覧

IM920::init(Pin tx_pin,Pin rx_pin,UartNumber uart_num)

初期化関数です。

パラメータ	説明
tx_pin, rx_pin	送受信ピン番号
uart_num	シリアル番号

```
IM920 im920;
```

```
int main(void){
```

```
    sken_system.init();
```

```
    im920.init(A0, A1, SERIAL4); // SERIAL4でIM920を初期化
```

```
    while(true){
```

```
    }
```

```
}
```

```
IM920::read(uint16_t* rx_id, uint8_t* rx_data, IM920Bytes bytes)
```

受信関数です。

パラメータ	説明
rx_id	受信先のノード番号を格納する変数アドレス
rx_data	受信データを格納する配列アドレス
bytes	受信データの大きさ (IM920Bytes)

```
IM920 im920;
```

```
uint16_t RxID;
```

```
uint8_t RxData[8];
```

```

int main(void){

    sken_system.init();

    im920.init(A0, A1, SERIAL4);


    while(true){

        im920.read(&RxID, RxData, Bytes8); // 8バイトで取得

    }

}

```

```

IM920::write(uint8_t* tx_data, IM920Bytes bytes, int node_num = 0x0000)

```

送信関数です。

パラメータ	説明
<code>tx_data</code>	送信データ配列のアドレス
<code>bytes</code>	送信データの大きさ (<code>IM920Bytes</code>)
<code>node_num</code>	送信先のノード番号 (省略時: 全ノード)

```

IM920 im920;

```

```

uint8_t TxData[8] = {0x91, 0xA2, 0xB3, 0xC4, 0xD5, 0xE6, 0xF7, 0x08};

```

```

int main(void){

    sken_system.init();

    im920.init(A0, A1, SERIAL4);

```

```
im920.write(TxData, Bytes8); // 8バイトのデータを全ノードに送信
```

```
while(true){  
  
}  
  
}
```

11. I2C クラス (I2C)

I2C通信を行います。

メソッド一覧

```
void I2C::init(Pin scl,Pin sda,I2CSelect i2c,int clockspeed)
```

I2C通信の初期化を行います。

パラメータ	説明
<code>scl, sda</code>	SCLピン, SDAピン
<code>i2c</code>	I2C番号
<code>clockspeed</code>	クロック速度 [Hz]

```
I2C i2c;
```

```
int main(void){
```

```
    sken_system.init();
```

```
    i2c.init(B8, B9, I2C_1, 400000); // I2C1, 400kHzで初期化
```

```
    while(true){
```

```
}  
  
}
```

```
void I2C::write(uint16_t DevAddress, uint16_t MemAddress,  
uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t  
Timeout = 100)
```

送信関数です。(HALのMemory Write相当)

パラメータ	説明
DevAddress	デバイスアドレス
MemAddress	メモリーアドレス
MemAddSize	メモリーアドレスサイズ
pData	送信データアドレス
Size	送信データサイズ
Timeout	タイムアウト時間 [ms] (デフォルト100ms)

```
I2C i2c;
```

```
uint8_t data[8] = {1, 2, 3, 4, 5, 6, 7, 8};
```

```
int main(void){
```

```
    sken_system.init();
```

```
    i2c.init(B8, B9, I2C_1, 400000);
```

```
    while(true){
```

```
        // デバイス0x01, メモリアドレス0xA0へ8バイト送信
```

```
        i2c.write(0x01, 0xA0, data, 8);
```

```
}  
  
}
```

```
void I2C::read(uint16_t DevAddress, uint16_t MemAddress,  
uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t  
Timeout = 100)
```

受信関数です。(HALのMemory Read相当)

パラメータ	説明
DevAddress	デバイスアドレス
MemAddress	メモリーアドレス
MemAddSize	メモリーアドレスサイズ
pData	受信データアドレス
Size	受信データサイズ
Timeout	タイムアウト時間 [ms] (デフォルト100ms)

```
I2C i2c;
```

```
uint8_t data[8];
```

```
int main(void){
```

```
    sken_system.init();
```

```
    i2c.init(B8, B9, I2C_1, 400000);
```

```
    while(true){
```

```
        // デバイス0x01, メモリアドレス0xA0から8バイト受信
```

```
        i2c.read(0x01, 0xA0, data, 8);
```

```
}  
  
}
```

12. 📌 定数・列挙体リファレンス (Constants and Enumerations)

本ライブラリで使用される、ハードウェアリソースやモード指定に関する定数と列挙体の一覧です。

1. ピン番号 (Pin / PIN)

定数グループ	説明	値の例
Pin	マイコンのGPIOピンを抽象化する列挙体。	A0 ~ A15 (GPIOA), B0 ~ B15 (GPIOB), C0 ~ C15 (GPIOC) など

// 使用例:

```
sken_system.startCanCommunicate(B9, B8, CAN_1); // B9ピンをCAN_TX、B8ピンをCAN_RXに設定
```

2. CAN通信選択 (CAN_SELECT)

定数	説明
CAN_1	1番目のCANコントローラ (CAN1) を選択
CAN_2	2番目のCANコントローラ (CAN2) を選択

// 使用例:

```
sken_system.startCanCommunicate(B9, B8, CAN_1);
```

3. UART/USART番号 (UartNumber)

定数	説明
SERIAL1	USART1を使用
SERIAL2	USART2を使用
SERIAL3	USART3を使用

定数	説明
SERIAL4	UART4を使用
SERIAL5	UART5を使用
SERIAL6	USART6を使用

// 使用例:

```
serial.init(A9, A10, SERIAL1, 9600);
```

4. タイマー番号 (TimerNumber)

定数	説明
TIMER1	タイマー1を使用
TIMER2	タイマー2を使用
TIMER3	タイマー3を使用
TIMER4	タイマー4を使用
TIMER5	タイマー5を使用
TIMER8	タイマー8を使用

// 使用例:

```
encoder.init(A0, A1, TIMER2); // タイマー2をエンコーダモードに設定
```

5. タイマーチャンネル (TimerChannel / CH)

定数	説明
CH1	タイマーチャンネル1
CH2	タイマーチャンネル2
CH3	タイマーチャンネル3
CH4	タイマーチャンネル4

// 使用例:

```
gpio.init(A5, PWM_OUTPUT, TIMER2, CH1);
```

6. I2C通信選択 (I2CSelect)

定数	説明
I2C_1	1番目のI2Cコントローラ (I2C1) を選択
I2C_2	2番目のI2Cコントローラ (I2C2) を選択
I2C_3	3番目のI2Cコントローラ (I2C3) を選択

// 使用例:

```
i2c.init(B8, B9, I2C_1, 400000);
```

7. IM920データサイズ (IM920Bytes)

定数	説明
Bytes8	8バイトのデータサイズ
Bytes16	16バイトのデータサイズ
Bytes24	24バイトのデータサイズ
Bytes32	32バイトのデータサイズ

// 使用例:

```
im920.write(TxData, Bytes8);
```

8. モータードライバーピン (MtrPin)

定数	説明
Apin	アルタイルドライバーのAピン側 (正転/逆転のどちらかのPWM)
Bpin	アルタイルドライバーのBピン側 (正転/逆転のどちらかのPWM)

// 使用例:

```
motor.init(Apin, A0, TIMER5, CH1);
```